AgileFlow AI

Human-in-the-Loop Multi-Agent Framework for Agile SDLC Automation

File 09 - Frontend Implementation and Centralized Theme Guide

Professional guide for React frontend structure, dark purple theme, reusable components, Figma-to-code handoff, and UI implementation rules.

Project	AgileFlow AI - Human-in-the-Loop Multi-Agent Framework for Agile SDLC Automation
Purpose	Guide the implementation team to build a consistent frontend using the approved UI/Figma direction.
Target Team	Implementation foundation team and frontend developers of all component teams
Primary Stack	React + Vite + Tailwind CSS + REST API services
Theme Direction	Professional dark SaaS interface with dark purple, violet, blue and neutral contrast

1. Purpose of This Guide

This document defines how the AgileFlow AI frontend should be implemented after the Figma design team completes the visual prototype. It protects the project from mixed UI styles, duplicated components, inconsistent spacing, different status colors, broken navigation, and frontend-backend mismatch.

The frontend must behave like a professional web application prototype. It should clearly represent the Agile SDLC workflow, human approval gates, central orchestrator, shared knowledge base, agent outputs, traceability, and validation reports.

Main principle

Build the frontend as a reusable design system first. Do not build each page independently with separate styles.

2. Recommended Frontend Stack

Area	Technology	Reason
Frontend framework	React + Vite	Fast setup, component-based development, suitable for research prototype dashboards
Styling	Tailwind CSS + CSS variables	Centralized theme tokens, fast implementation, consistent reusable components
Routing	React Router	Clean page routing for dashboard, agents, orchestrator and reports
API calls	Axios or Fetch wrapper	One shared API client to avoid repeated request logic
State	React state first, optional Zustand later	Keep MVP simple; add global state only where useful
Forms	Controlled components or React Hook Form	Consistent validation and clean form structure
Charts/diagrams	Simple CSS/SVG first	Avoid heavy libraries unless needed for demo

3. Frontend Folder Structure

Use one approved structure from the beginning. Every member should place their pages, components, services, and mock data in the correct location.

```
frontend/
├── src/
│   ├── app/              # App.jsx and routes.jsx
│   ├── assets/
│   ├── components/       # common, layout, workflow, approvals, tables, reports, feedback
│   ├── pages/            # dashboard, projects, requirements, design, testing, implementation, orchestrator, reports
│   ├── services/         # shared API client and service files
│   ├── styles/           # tokens.css, theme.js, globals.css
│   ├── constants/
│   ├── hooks/
│   ├── utils/
│   └── mock-data/
```

Conflict prevention rule

Members must not create random folders such as ui2, finalUI, new-components, agentScreens, or temp-pages. All work must follow this structure.

4. Centralized Dark Purple Theme

The implementation must use a centralized theme. No member should hardcode random colors inside pages. Colors should come from tokens.css, Tailwind config, or a shared theme object.

4.1 Core Color Palette

Token	Hex	Use	Guideline
bg-main	#0B0714	Application background	Use for main body background
bg-sidebar	#120A24	Sidebar and navigation	Keep sidebar visually stable
bg-card	#1A102F	Cards and panels	Primary content surface
bg-card-soft	#21153A	Secondary panels	For nested content only
border-soft	#342454	Borders/dividers	Use subtle borders
primary-purple	#7C3AED	Primary actions	Main CTA buttons and active states
accent-violet	#A855F7	Highlights	Badges, focus, selected stage
accent-blue	#38BDF8	Information	Links and info indicators
text-primary	#F8FAFC	Primary text	Headings and important labels
text-secondary	#CBD5E1	Secondary text	Descriptions and helper text
text-muted	#94A3B8	Muted text	Metadata and disabled text
success	#22C55E	Approved/completed	Positive status only
warning	#F59E0B	Pending/waiting	Approval waiting or attention
error	#EF4444	Failed/rejected	Errors, rejected approvals, failed tests

4.2 Status Color Rules

Status	Color token	Usage
PENDING	text-muted / border-soft	Not started yet
IN_PROGRESS	accent-blue	Currently processing
WAITING_FOR_APPROVAL	warning	Human review needed
APPROVED	success	Accepted by human reviewer
REJECTED	error	Rejected by reviewer
FAILED	error	Build/test/security/workflow failed
COMPLETED	success	Stage successfully completed

4.3 Theme Implementation Note

Store these values in frontend/src/styles/tokens.css and map them into Tailwind or shared class names. Pages should use tokens, not random hardcoded color values.

5. Typography, Spacing and Shape Rules

5.1 Typography Scale

Use	Font size	Weight	Example location
Page title	28-32px	700	Dashboard title, project title
Section heading	20-24px	600/700	Agent page sections
Card title	16-18px	600	Reusable cards and panels
Body text	14-16px	400/500	Descriptions and table content
Helper text	12-13px	400	Metadata and form helper messages

5.2 Spacing, Radius and Shadows

- Use an 8px spacing system: 4, 8, 12, 16, 24, 32, 48.
- Main page padding: 24px on desktop, 16px on tablet/mobile.
- Card padding: 20px or 24px depending on content density.
- Button height: 40px minimum; important buttons: 44px.
- Border radius: 12px for cards, 10px for buttons, 8px for inputs and badges.
- Use subtle shadows only. The dark theme should look premium, not glowing or decorative.

6. Centralized Layout Plan

The application should use one shared layout for all pages. Individual agent pages should not build their own sidebar, topbar, or navigation layout.

```
MainLayout
├── Sidebar
├── Topbar
├── PageHeader
└── ContentArea
    ├── Page content
    ├── Approval gate card
    └── Activity / artifact panels
```

6.1 Sidebar Navigation

- Dashboard
- Projects
- Requirement Gathering and Analysis
- Project Management and Design
- Pre-Code Test Planning
- Implementation and Git Automation
- Testing, Validation and Security
- Orchestrator
- Traceability
- Reports
- Settings

6.2 Topbar Content

- Current project selector
- Workflow status badge
- Search placeholder
- Notifications area
- User/profile placeholder

7. Page and Route Plan

Routes must be predictable and aligned with the system workflow. Use nested routes where needed, but keep the MVP simple.

Page	Route	Owner	Purpose
Dashboard	/dashboard	Foundation team	Project summary, pending approvals, workflow overview
Projects	/projects	Foundation team	Project list and create project entry point
Project Workflow	/projects/:projectId/workflow	Orchestrator/foundation	Full SDLC workflow tracker
Requirement Gathering and Analysis	/projects/:projectId/requirements	Requirement component	Client inputs, requirements, SRS, H1 approval
Project Management and Design	/projects/:projectId/design	Design component	Backlog, sprint plan, API/DB/UI plan, H2 approval
Pre-Code Test Planning	/projects/:projectId/testing/pre-code	Testing component	Test specs, test skeletons, H3 approval
Implementation and Git Automation	/projects/:projectId/implementation	Implementation component	Code modules, Git metadata, logs, H4 approval
Testing, Validation and Security	/projects/:projectId/testing/validation	Testing component	Execution results, bug/security reports, H5 approval
Orchestrator	/projects/:projectId/orchestrator	Foundation/orchestrator	Workflow control, routing, retry loops
Traceability Report	/projects/:projectId/traceability	Reports team	Requirement to design to code to test links
Final Reports	/projects/:projectId/reports	Reports team	SRS, design, test, bug, security and validation reports

8. Reusable Component Plan

Reusable components should be implemented before full pages. Component reuse keeps the UI consistent and reduces merge conflicts.

Component	Folder	Used by	Responsibility
MainLayout	components/layout	All pages	Sidebar, topbar and content frame
PageHeader	components/common	All pages	Title, subtitle, actions and breadcrumbs
StatusBadge	components/common	All pages	Consistent stage/status display
AgentCard	components/workflow	Dashboard/workflow	Displays each component and its current state
WorkflowStepper	components/workflow	Project workflow	Shows stage sequence and approval gates
ApprovalGateCard	components/approvals	All approval pages	Approve, reject, request changes, add comment
DataTable	components/tables	All list pages	Reusable table with empty/error/loading states
TraceabilityTable	components/tables	Traceability/report pages	Requirement to design to

			code to test mapping
ArtifactPanel	components/common	Agent pages	Preview generated outputs
FileUploadBox	components/common	Requirement input page	Drag/drop or file selection placeholder
LogViewer	components/feedback	Implementation/testing pages	Build/runtime/test/security logs
ReportCard	components/reports	Reports page	Summary of generated report sections
ActivityTimeline	components/feedback	Dashboard/orchestrator	Workflow events and audit trail

Reusable component rule

If two pages need the same UI pattern, create a component. Do not duplicate the same card, table, approval block or status badge inside multiple pages.

9. API Integration Rules

All API calls must go through service files. Page components should not directly contain raw fetch or axios logic except for temporary experiments.

```
services/
├── apiClient.js
├── projectService.js
├── requirementService.js
├── designService.js
├── testingService.js
├── implementationService.js
├── orchestratorService.js
└── reportService.js
```

9.1 API Client Rules

- Use one base API URL from environment variables.
- Handle common request headers in one place.
- Normalize error responses before showing them to UI components.
- Keep loading, error, and empty states visible in the UI.
- Do not hardcode project IDs in services or pages.

9.2 Response Shape Expected by Frontend

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": {},
  "errors": [],
  "metadata": {
    "projectId": "PRJ-001",
    "workflowStage": "H1_WAITING_APPROVAL"
  }
}
```

10. Page-Level Implementation Rules

- Every page must show a clear page title, short description, current project context, and current workflow state.
- Every page that depends on backend data must include loading, error, and empty states.
- Every approval page must use ApprovalGateCard instead of custom buttons.
- Every generated artifact must show its source stage and traceability ID when available.
- Every table must use the shared DataTable pattern.
- Each agent page should contain input area, generated output area, approval area, and activity/history area where relevant.

11. Human Approval UI Rules

Gate	UI label	Action rules
H1	Requirement Approval	Approve, Reject, Request Changes, Comment
H2	Design and Planning Approval	Approve design package before test planning
H3	Pre-Code Test Approval	Approve test specifications before implementation
H4	Implementation Approval	Approve generated prototype before validation
H5	Final Validation Approval	Approve reports and final evidence

Approval rule

Approval buttons must be disabled until required artifacts are available. The UI must clearly show who approved, when approved, and what comment was given.

12. Figma-to-Code Handoff Rules

The Figma design team and implementation team must work from one shared UI agreement. Screenshots alone are not enough; the implementation team also needs tokens, components, sizes, and interaction notes.

12.1 What the UI Team Must Provide

- Final page list and navigation flow
- Desktop frames for each main page
- Reusable component frames
- Color tokens and status colors
- Typography scale and spacing rules
- Button, input, badge, table and card variants
- Approval gate component design
- Workflow stepper design
- Traceability table design
- Empty, loading and error state designs

12.2 What Implementation Team Must Not Do

- Do not change the visual direction without informing UI team.
- Do not create new colors outside the theme unless approved.
- Do not rename components differently from the agreed component list.
- Do not skip empty/error/loading states.
- Do not implement pages before the shared layout and theme are ready.

13. Accessibility and Responsiveness

- Text contrast must be readable on dark purple backgrounds.
- Buttons and clickable controls should be at least 40px high, preferably 44px.
- Use visible focus states for keyboard users.
- Do not use color only to communicate state; include labels such as Approved, Failed, Pending.
- Tables should scroll horizontally on smaller screens instead of breaking layout.
- Sidebar can collapse on smaller screens after MVP if time permits.
- Forms must show clear validation messages below fields.

14. Frontend Quality Checklist

Checklist item	Required before PR?
Uses approved route and folder structure	Yes
Uses centralized theme tokens	Yes
Uses reusable components	Yes
Has loading, error and empty states	Yes
Uses correct workflow status labels	Yes
Does not hardcode project IDs	Yes
Does not duplicate an existing component	Yes
Responsive at basic desktop and tablet widths	Yes
Screens match Figma direction closely	Yes
API calls use service layer	Yes

15. AI Coding Agent Prompt Template for Frontend Tasks

When using AI coding tools, every member must provide strict scope. The prompt below prevents the AI agent from editing unrelated files or creating duplicate UI patterns.

```
You are working on AgileFlow AI frontend.

Task:
[Describe the exact frontend task]

Allowed files/folders:
[List only allowed folders]

Do not modify:
- Shared routes unless required
- Existing theme tokens without approval
- Other agent pages
- Backend files

Follow these documents:
- File 03 System Architecture and Component Relationship Summary
- File 06 API and Data Schema Plan
- File 07 Task Assignment and Branching Rules
- File 09 Frontend Implementation and Centralized Theme Guide

Frontend rules:
- Use React + Vite structure.
- Use centralized theme tokens.
- Reuse existing components where possible.
- Add loading, error and empty states.
- Do not hardcode project IDs.
- Return a summary of changed files.
```

16. First Frontend Implementation Order

1. Create frontend project using Vite and install required dependencies.
2. Add Tailwind CSS or centralized CSS token setup.
3. Create tokens.css, theme.js and globals.css.
4. Create MainLayout with Sidebar and Topbar.
5. Create basic route structure.
6. Create common components: PageHeader, StatusBadge, DataTable, Button, Card.
7. Create workflow components: WorkflowStepper, AgentCard, ApprovalGateCard.
8. Create Dashboard and Projects pages.
9. Create placeholder pages for all agent modules.
10. Connect service layer to backend when foundation APIs are ready.
11. Replace placeholder content with real agent screens one by one.

17. Final Rule

Frontend implementation rule

First build the design system and layout. Then build pages. Then connect APIs. This order is safer than building all pages separately and trying to fix consistency later.

This guide should be uploaded to the implementation team chat together with the API plan, implementation working plan, task assignment document, and the final Figma design export. It becomes the main frontend agreement for all component teams.